Veterinarian Portal API Reference

This document defines the veterinarian-facing endpoints delivered in Sprint 6: pre-approval onboarding, the personal appointment queue, completing appointments, vaccination records and the vaccine catalogue, health records, the universal health passport, and account closure. It follows the same conventions established in 07-Public API Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here. Roles are listed per endpoint. A veterinarian is identified via `req.user.userID`; their shelter is resolved server-side and is never read from the request body. All example data in this document is fictional.

1. Overview

The veterinarian portal is the clinical workspace of PetPals. Registration and login are documented in 06-Auth_System; this document covers everything a veterinarian does after that point. It spans eight areas:

- **Onboarding before approval** — a new vet completes a short wizard (profile, address, government ID) while still Pending (Sections 3 and 4).
- **Appointment queue** — a vet sees only the appointments assigned to them, and may reschedule or complete them (Section 5).
- **Completing appointments** — marking an appointment Completed with optional clinical notes that become a health record (5.3).
- **Vaccinations** — recording doses given at an appointment, the overdue worklist, and treatment stats (Section 6).
- **Vaccine catalogue** — one network-wide list of vaccines that vets and admins can extend and correct (6.1–6.3).
- **Health records and the universal health passport** — standalone notes, and a pet's full history across every shelter it has been to (Section 7).
- **AI on the health passport** — "Summarize with AI": an AI summary of a pet's passport with cited points to review (7.6).
- **Closing the account** — deactivate or permanently delete, with history preserved (4.7 and Section 9).

The table below lists every endpoint a veterinarian can call. Paths are relative to `/api/v1`. The Pending column shows whether a Pending vet may call the route (see Section 2). Endpoints shared with other roles are marked with the roles that may use them; only the veterinarian behaviour of shared endpoints is specified here, and the Staff and Admin behaviour remains in 11-Staff API Reference and 10-Admin API Reference.

Endpoint	Roles	Pending	Purpose
GET /vets/me	Vet	Yes	Own profile (4.1)
PUT /vets/me	Vet	Yes	Update own profile (4.2)
GET /vets/me/government-id	Vet	Yes	Government ID status, masked (4.3)
POST /vets/me/government-id	Vet	Yes	Submit or resubmit government ID (4.4)
PATCH /vets/me/onboarding-step	Vet	Yes	Advance the onboarding wizard (4.5)
PATCH /vets/me/onboarding-complete	Vet	Yes	Finish onboarding (4.6)
DELETE /vets/me	Vet	No	Deactivate or delete own account (4.7)
GET /vets/me/appointments	Vet	No	Own appointment queue (5.1)
GET /vets/me/appointments/:id	Vet	No	One appointment in detail (5.2)

Endpoint	Roles	Pending	Purpose
PATCH /appointments/:id/status	Vet (assigned)	No	Complete an appointment (5.3)
PATCH /appointments/:id	Staff, Admin, Vet (assigned)	No	Reschedule date and reason (5.4)
GET /vaccines	Admin, Staff, Vet	No	Vaccine catalogue (6.1)
POST /vaccines	Vet, Admin	No	Add a vaccine (6.2)
PUT /vaccines/:id	Vet, Admin	No	Edit a vaccine (6.3)
POST /appointments/:id/vaccinations	Vet (assigned)	No	Record a dose (6.4)
GET /appointments/:id/vaccinations	Vet (assigned), Staff, Admin	No	Doses given at an appointment (6.5)
GET /vets/me/vaccinations/overdue	Vet	No	Overdue worklist for the shelter (6.6)
GET /vets/me/stats	Vet	No	Pets treated (6.7)
GET /vets/me/pets	Vet	No	Pets at the vet's shelter (7.1)
GET /vets/me/pets/:id	Vet	No	One pet in detail (7.2)
GET /vets/me/pets/:id/health-passport	Vet	No	Universal health passport (7.3)
GET /pets/:id/health-passport/summary	Staff, Admin, Vet	No	AI passport summary (7.6)
POST /pets/:id/health-records	Vet	No	Add a standalone health note (7.4)
PUT /health-records/:id	Vet (author)	No	Edit own health note (7.5)

> ***Never returned:*** *userPassword, refreshToken, the full government ID number (always masked) and stripeCustomerID are excluded from every response in this document, regardless of role — this mirrors the NF-10 data privacy requirement.*

2. Vet Lifecycle & Access Rules

A veterinarian moves through three account states. Unlike Volunteers and Admins, a Pending vet is allowed to log in, so that onboarding can happen before approval.

State	Can log in	API access	How the vet gets here
Pending	Yes	Only the allowPending routes in 2.2; every other route returns 401	Registration (2.1)
Active	Yes	All veterinarian routes	Approved by the shelter's Manager (2.3)
Deactivated	No — rejected at login	None; an already-issued access token stops working immediately (2.4)	Declined or deactivated by the shelter's Manager (2.3), or self-deactivated (4.7)

2.1 Registration

A vet registers through the standard registration endpoint (see 06-Auth_System) with role `"vet"` and a `shelterID`. Registration is refused with VALIDATION_ERROR (422) if the chosen shelter has no manager, because a Pending vet can only be approved by their shelter's Manager and would otherwise have nobody able to approve them. A successful registration creates the account in the Pending state.

2.2 Pending vets and allowPending routes

A Pending vet can log in. The login and refresh responses include `accountStatus`, `onboardingComplete` and `onboardingStep`, which the frontend uses to send the vet to the right place (the wizard, or an awaiting-approval screen). Routes opt in to Pending access through the `authenticate.allowPending` middleware. A Pending vet may reach only the following routes:

Route	Section
GET /vets/me and PUT /vets/me	4.1, 4.2
GET /vets/me/government-id and POST /vets/me/government-id	4.3, 4.4
PATCH /vets/me/onboarding-step	4.5
PATCH /vets/me/onboarding-complete	4.6
POST /auth/logout	06-Auth_System

Every other route returns UNAUTHORIZED (401) for a Pending vet — including DELETE /vets/me, so a Pending vet cannot close their account until they are approved or declined. Pending Volunteer and Admin accounts are different: they still cannot log in at all and receive 401 at login.

> ***Why 401 and not 403:*** *the frontend redirects every 403 response to /forbidden. A Pending vet is not forbidden from the portal — they simply are not approved yet — so a 403 would send them to the wrong page. 401 keeps them on the login/approval flow. This is deliberate and should not be "corrected" to 403.*

A Pending-vet rejection looks like any other error response:

```
{
  "success": false,
  "message": "Your account is awaiting approval",
  "error": { "code": "UNAUTHORIZED", "details": "..." }
}
```

2.3 Approval and decline

Approval belongs to the vet's shelter Manager, not to Admins: `PATCH /api/v1/staff/me/vets/:id/status` with `{ accountStatus: "Active" }` (see 11-Staff API Reference, 11.5). The rules are:

- **Approving** a Pending vet returns CONFLICT (409) unless onboarding is complete **and** the government ID is Verified. The error message names whatever is missing (for example, that the government ID is Pending, not Verified).
- **Declining** (Pending → Deactivated) is allowed at any stage, including before onboarding is finished.
- **Deactivating** an Active vet (Active → Deactivated) is also done by the Manager.
- The government ID itself is reviewed by the same Manager through the government ID review endpoints (11-Staff API Reference, Section 10): a shelter's Manager reviews its Veterinarians' IDs.

2.4 Live account-status enforcement

Account status is checked live on every protected request, not only at login. Approval, decline and deactivation therefore take effect immediately without the vet logging in again: a newly approved vet's existing session starts working, and a deactivated vet's already-issued access token is rejected on its next use.

2.5 Existence masking: 404 instead of 403

A vet must not be able to discover which appointments exist, or who they belong to. Where a resource belongs to a different vet, the API answers exactly as if it did not exist.

Situation	Response
Another vet's appointment on any appointment route (5.2, 5.3, 5.4, 6.4, 6.5)	404 NOT_FOUND — identical to an appointment that does not exist
A pet at another shelter on the vet read routes (7.2, 7.3, 7.6)	404 NOT_FOUND
A pet at another shelter on the write route POST /pets/:id/health-records (7.4)	403 FORBIDDEN
Editing a health record the vet did not write, or one with no vet (7.5)	403 FORBIDDEN
Staff or Admin calling a vet-only write (5.3, 6.4)	403 FORBIDDEN
A Pending vet calling a route that is not allowPending	401 UNAUTHORIZED

3. Onboarding Wizard

A new vet completes a four-step wizard while still Pending. Registration counts as step 1 and happens before the wizard. At each step the client first saves that step's data, then calls `PATCH /vets/me/onboarding-step` with the step just completed. The client resumes the wizard from the `onboardingStep` returned at login.

Step	Screen	Data saved first	Then call
1	Registration	Account created at registration (not part of the wizard)	—
2	Personal	vetName, vetPhone, vetDOB, vetSex, avatarSeed via PUT /vets/me (4.2)	PATCH /vets/me/onboarding-step { step: 2 }
3	Address	addressLine1, addressLine2, city, state, zip, country via PUT /vets/me (4.2)	PATCH /vets/me/onboarding-step { step: 3 }
4	Identity	Government ID via POST /vets/me/government-id (4.4)	PATCH /vets/me/onboarding-step { step: 4 }
5	Review	Nothing new — the vet reviews and submits	PATCH /vets/me/onboarding-complete (4.6), which sets onboardingStep to 5

`onboardingStep` only ever advances: the server sets it to the larger of its current value and `step + 1`, capped at 5, so going back in the wizard or repeating a call never moves the vet backward.

Required for completion: vetPhone, vetDOB, vetSex, addressLine1, city, state, zip, country, and a submitted government ID. addressLine2 is optional. Once onboarding is complete the vet waits for the Manager's approval, which additionally requires the government ID to be Verified (2.3).

4. Self-Service Endpoints

These endpoints let the logged-in veterinarian manage their own profile, onboarding and account. The first six are allowPending: they work while the vet is still awaiting approval. All of them identify the vet via `req.user.userID`.

4.1 GET /api/v1/vets/me

Returns the logged-in vet's own profile, including their shelter's contact details.

Request

Field	Value
Method	GET

Field	Value
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Path/Body fields	None — vet identified via req.user.userID

Example request

```
GET /api/v1/vets/me
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "avatarSeed": "a1b2c3d4-...",
    "vetName": "Dr. Jay Asarathi",
    "vetPhone": "+15555550142",
    "vetDOB": "1985-11-02",
    "vetSex": "M",
    "addressLine1": "100 Example Street",
    "addressLine2": null,
    "city": "Athens",
    "state": "GA",
    "zip": "30601",
    "country": "USA",
    "shelter": {
      "shelterName": "Athens Shelter",
      "shelterAddress": "200 Sample Avenue, Athens, GA",
      "shelterPhone": "+15555550100",
      "shelterEmail": "info@athens-shelter.example"
    },
    "accountStatus": "Active",
    "onboardingComplete": true,
    "onboardingStep": 5,
    "createdAt": "2026-07-14T10:00:00Z",
    "emailVerified": true,
    "lastLoginAt": "2026-10-03T08:30:00Z",
    "user": { "userEmail": "jay.asarathi@example.com" }
  }
}
```

`emailVerified` and `lastLoginAt` come from the USERS table, not the VETERINARIAN table. `shelter` is null if the vet has no shelter assigned. Passwords and refresh tokens are never included.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Veterinarian record does not exist	NOT_FOUND	404

4.2 PUT /api/v1/vets/me

Partially updates the logged-in vet's own profile. Only fields present in the body are changed.

Request

Field	Value
Method	PUT
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Body fields	Any subset of: avatarSeed, vetName, vetPhone, vetDOB (date), vetSex (M / F / O), addressLine1, addressLine2, city, state, zip, country. vetPhone, vetDOB and vetSex accept null to clear them; no other field is nullable.

vetPhone is normalised to E.164 server-side; a number that cannot be parsed is rejected with VALIDATION_ERROR (422). shelterID and accountStatus are not editable here — providing either returns 400 listing the rejected field(s), before any write occurs (shelter and status changes are Manager/Admin-controlled).

Example request

```
PUT /api/v1/vets/me
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "vetPhone": "(555) 555-0142",
  "vetSex": "M",
  "city": "Athens"
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /vets/me (4.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Rejected field present (shelterID, accountStatus)	BAD_REQUEST	400
Invalid value (vetSex not M/F/O, invalid vetDOB, wrong type), or null on a field that is not nullable	BAD_REQUEST	400
No updatable fields provided	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Veterinarian record does not exist	NOT_FOUND	404
vetPhone cannot be parsed to E.164	VALIDATION_ERROR	422

4.3 GET /api/v1/vets/me/government-id

Returns the logged-in vet's submitted government ID, with the ID number masked.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Path/Body fields	None — vet identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Government ID retrieved successfully",
  "data": {
    "governmentIDID": 11,
    "userID": 7,
    "userType": "Veterinarian",
    "idType": "Driver's License",
    "idNumber": "*****4567",
    "verificationStatus": "Pending",
    "documentURL": "..."
  }
}
```

Response masking: idNumber is masked (e.g. *****4567) on this endpoint and on the POST response — the full number is never returned to the vet. Only the vet's shelter Manager sees the unmasked number, through the review endpoints (11-Staff API Reference, 10.2).

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
No government ID has been submitted yet	NOT_FOUND	404

4.4 POST /api/v1/vets/me/government-id

Submits a government ID for identity verification. The file is stored in the private `government-ids` Storage bucket; only metadata is persisted in the GOVERNMENT_ID table.

Request

Field	Value
Method	POST
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Content-Type	multipart/form-data
Body fields	idType (string, required, ≤45 chars), idNumber (string, required, ≤45 chars), file (JPEG/PNG/WebP/HEIC/PDF, ≤5 MB, required)

Example request

```
POST /api/v1/vets/me/government-id
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data

idType=Driver's License
idNumber=D1234567
file=@license-front.jpg
```

Response

```
201 Created
{
  "success": true,
  "message": "Government ID submitted successfully",
  "data": {
    "governmentIDID": 11,
    "userID": 7,
    "userType": "Veterinarian",
    "idType": "Driver's License",
    "idNumber": "*****4567",
    "verificationStatus": "Pending",
    "documentURL": "..."
  }
}
```

One per vet: uniqueness is the same `@@unique([userID, userType])` constraint used for every role. A second submission while the existing ID is Pending or Verified returns CONFLICT (409). A **Rejected** ID can be resubmitted: the existing row is overwritten in place and reset to Pending, and the previously stored file is removed from Storage.

Error Cases

Scenario	Error Code	HTTP Status
Missing idType, idNumber or file; idType/idNumber too long	BAD_REQUEST	400
File exceeds 5 MB or is not an accepted type	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
A government ID already exists and is not Rejected	CONFLICT	409

4.5 PATCH /api/v1/vets/me/onboarding-step

Advances the logged-in vet through the onboarding wizard. Called after a step's own data has been saved (see Section 3).

Request

Field	Value
Method	PATCH
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Body fields	step (int, required, 2–5) — the wizard step just completed

Advance-only: the server sets onboardingStep = min(max(currentStep, step + 1), 5). A lower or equal step is ignored rather than moving the vet backward, and the value is capped at 5.

Example request

```
PATCH /api/v1/vets/me/onboarding-step
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "step": 3 }
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding step updated successfully",
  "data": { "...": "same shape as GET /vets/me (4.1), onboardingStep: 4" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
step missing or not an integer from 2 to 5	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Veterinarian record does not exist	NOT_FOUND	404

4.6 PATCH /api/v1/vets/me/onboarding-complete

Marks onboarding complete — the final submit of the wizard's Review step.

Request

Field	Value
Method	PATCH
Auth required	Yes — Veterinarian only
Pending allowed	Yes (allowPending)
Body fields	None

Requires vetPhone, vetDOB, vetSex, addressLine1, city, state, zip, country and a submitted government ID (of any verification status). Sets onboardingComplete to true and onboardingStep to 5. A Pending vet then waits for the Manager's approval, which also requires the ID to be Verified (2.3).

Example request

```
PATCH /api/v1/vets/me/onboarding-complete
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding completed successfully",
  "data": { "...": "same shape as GET /vets/me (4.1)" }
```

```
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Veterinarian record does not exist	NOT_FOUND	404
One or more required items are missing — the message names every missing item	CONFLICT	409

Example 409 body when only the ID is missing:

```
{
  "success": false,
  "message": "Onboarding is incomplete — missing: Government ID",
  "error": { "code": "CONFLICT", "details": "..." }
}
```

4.7 DELETE /api/v1/vets/me

Closes the vet's own account, either by deactivating it (data retained) or permanently deleting it. See Section 9 for exactly what each mode keeps.

Request

Field	Value
Method	DELETE
Auth required	Yes — Veterinarian only
Pending allowed	No — a Pending vet gets 401
Body fields	mode (string, required — "deactivate" or "delete")

`deactivate` sets accountStatus to Deactivated and clears the refresh token. `delete` removes the Veterinarian and Users rows, plus the vet's government ID and its stored Storage file; the vet's history survives (Section 9). Both modes are blocked while the vet has **any future Scheduled appointment**: a vet cannot walk away from patients who are still booked with them.

Example request

```
DELETE /api/v1/vets/me
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "mode": "deactivate" }
```

Response

```
200 OK
{
  "success": true,
  "message": "Account deactivated",
  "data": null
}
```

The message is `"Account deleted"` when mode is `delete`.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
mode missing or not "deactivate"/"delete"	VALIDATION_ERROR	422
The vet has an upcoming Scheduled appointment (both modes)	CONFLICT	409

Example 409 body:

```
{
  "success": false,
  "message": "You have upcoming appointments — ask your shelter's staff to
reassign them before closing your account",
  "error": { "code": "CONFLICT", "details": "..." }
}
```

Staff reassign appointments with PATCH /appointments/:id (11-Staff API Reference, 4.4), which is how a vet's upcoming appointments are cleared before they leave.

5. Appointments

A vet works from their own queue: the appointments where `vetID` is the caller. Vets can reschedule and complete appointments assigned to them and record vaccinations at them (Section 6), but cannot create or cancel appointments (5.5). Appointment statuses are described in Section 8.

5.1 GET /api/v1/vets/me/appointments

Returns the logged-in vet's own appointment queue, paginated. A vet never sees another vet's appointments here, regardless of shelter.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Query params	upcoming (optional — strictly "true" or "false", default "false"), petName (optional — case-insensitive contains match), dateFrom / dateTo (optional ISO date-times, inclusive range on appointmentDate), page (int, optional, default 1), limit (int, optional, default 20, max 100)

`upcoming=true` returns Scheduled appointments dated in the future, soonest first. `upcoming=false` returns past-dated or Cancelled appointments, most recent first. Any value other than the exact strings "true" or "false" is rejected rather than ignored.

Example request

```
GET /api/v1/vets/me/appointments?upcoming=true&petName=bud&page=1&limit=20
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Appointments retrieved successfully",
  "data": [
    {
      "appointmentID": 123,
      "appointmentDate": "2026-10-05T10:00:00Z",
      "appointmentReason": "Annual checkup",
      "status": "Scheduled",
      "pet": {
        "petID": 9,
        "petName": "Buddy",
        "petPhoto": "...",
        "breedName": "Beagle",
        "speciesName": "Dog"
      },
      "vetName": "Dr. Jay Asarathi"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

The list item has the same shape as Staff GET /appointments (11-Staff API Reference, 4.1). `status` is the displayed status (Section 8.1).

Error Cases

Scenario	Error Code	HTTP Status
upcoming is not "true"/"false"; invalid dateFrom/dateTo (or dateFrom after dateTo); invalid page or limit	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

5.2 GET /api/v1/vets/me/appointments/:id

Returns one of the vet's own appointments in full, including the vaccines administered and any health records written at it.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only (own appointments)
Pending allowed	No
Path params	id (int, required) — appointmentID

Example request

```
GET /api/v1/vets/me/appointments/120
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Appointment retrieved successfully",
  "data": {
    "appointmentID": 120,
    "appointmentCode": "APT-00120",
    "appointmentDate": "2026-09-28T10:00:00Z",
    "appointmentReason": "Annual checkup",
    "status": "Completed",
    "appointmentStatus": "Scheduled",
    "pet": {
      "petID": 9,
      "petName": "Buddy",
      "petPhoto": "...",
      "breedName": "Beagle",
      "speciesName": "Dog"
    },
    "vetName": "Dr. Jay Asarathi",
    "staffName": "Casey Example",
    "volunteerName": null,
    "shelterName": "Athens Shelter",
    "shelterAddress": "200 Sample Avenue, Athens, GA",
    "vaccinesAdministered": [
      {
        "recordID": 31,
        "vaccineName": "Rabies",
        "administeredDate": "2026-09-28T00:00:00Z",
        "dueDate": "2027-09-28T00:00:00Z"
      }
    ],
    "healthRecords": []
  }
}
```

Two status fields: `status` is the displayed status; `appointmentStatus` is the stored value. In the example the appointment is past-dated but was never formally completed, so it displays as Completed while the stored status is still Scheduled (Section 8.1). Clients decide whether to offer the "Complete" action from `appointmentStatus`, not `status`.

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Appointment does not exist, or belongs to another vet (indistinguishable by design)	NOT_FOUND	404

5.3 PATCH /api/v1/appointments/:id/status

Completes an appointment. This is the only real write of the Completed status — nothing else sets it. Optionally records clinical notes as a health record in the same operation.

Request

Field	Value
Method	PATCH
Auth required	Yes — Veterinarian only (assigned vet)
Pending allowed	No
Path params	id (int, required) — appointmentID
Body fields	appointmentStatus (string, required — only "Completed" is accepted), notes (string, optional, ≤500 chars)

Completion rules

- The **stored** appointmentStatus must be Scheduled. A stored Completed or Cancelled appointment returns 409 with a message of the form "A <status> appointment can't be completed". A past-dated Scheduled appointment that merely displays as Completed can still be completed — it is stored as Scheduled.
- An appointment cannot be completed before its `appointmentDate`; attempting it returns 409.
- The write is conditional on the stored status still being Scheduled. If another request (for example, a Staff cancellation) wins the race, this request returns 409 "This appointment is no longer Scheduled" and changes nothing.
- If `notes` is present and not blank, a HealthRecord is created in the **same transaction**, linked to the appointment via appointmentID, with the vet as author. Blank or whitespace-only notes create nothing. If either write fails, neither is applied.

Example request

```
PATCH /api/v1/appointments/120/status
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "appointmentStatus": "Completed",
  "notes": "Healthy weight, no concerns. Rabies booster given."
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Appointment completed successfully",
  "data": {
    "...": "same shape as GET /vets/me/appointments/:id (5.2)",
    "status": "Completed",
    "appointmentStatus": "Completed"
  }
}
```

The returned healthRecords array includes the note just written, for example { "recordID": 18, "recordDesc": "Healthy weight, no concerns. Rabies booster given.", "createdAt": "2026-09-28T11:05:00Z" }.

Error Cases

Scenario	Error Code	HTTP Status
Body invalid: appointmentStatus missing or not "Completed", notes not a string or over 500 chars, or id not a positive integer	BAD_REQUEST	400

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Caller is Staff or Admin	FORBIDDEN	403
Appointment does not exist, or is assigned to another vet	NOT_FOUND	404
Stored status is not Scheduled ("A <status> appointment can't be completed")	CONFLICT	409
appointmentDate has not been reached yet	CONFLICT	409
Lost a race — no longer Scheduled when the write ran	CONFLICT	409

Example 409 body:

```
{
  "success": false,
  "message": "A Cancelled appointment can't be completed",
  "error": { "code": "CONFLICT", "details": "..." }
}
```

5.4 PATCH /api/v1/appointments/:id

Staff's appointment edit route (11-Staff API Reference, 4.4), which is also open to the assigned vet for two fields only.

Request

Field	Value
Method	PATCH
Auth required	Yes — Staff, Admin, Veterinarian (assigned vet; restricted fields)
Pending allowed	No
Path params	id (int, required) — appointmentID
Body fields	Any subset of: appointmentDate (date-time, not in the past), appointmentReason (string, ≤300). At least one is required.

For a vet, vetID, staffID and volunteerID are not editable — sending any of them returns 400 (reassigning people is Staff work). Only a Scheduled appointment whose date has not passed can be edited. The double-booking guard applies, excluding this appointment itself: no two Scheduled appointments may share the same pet, vet and time.

Example request

```
PATCH /api/v1/appointments/123
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "appointmentDate": "2026-10-07T14:00:00Z",
  "appointmentReason": "Annual checkup and booster"
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Appointment updated successfully",
```

```
  "data": { "...": "same shape as GET /vets/me/appointments/:id (5.2)" }
}
```

The vet response never includes adopter details.

Error Cases

Scenario	Error Code	HTTP Status
No fields, invalid field, appointmentDate in the past, appointmentReason over 300 chars, or vetID/staffID/volunteerID present	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Appointment does not exist, or is assigned to another vet	NOT_FOUND	404
Appointment is past, or no longer Scheduled	CONFLICT	409
The new slot is already booked for this pet and vet	CONFLICT	409

5.5 Actions not available to vets

Two appointment actions are deliberately closed to the Veterinarian role. Both return FORBIDDEN (403) for a vet.

Action	Endpoint	Who can do it
Cancel an appointment	PATCH /appointments/:id/cancel	Staff, Admin (11-Staff API Reference, 4.5)
Create an appointment	POST /appointments	Staff, Admin (11-Staff API Reference, 4.2)

A vet who cannot attend an appointment asks the shelter's staff to cancel or reassign it.

6. Vaccinations & Vaccine Catalogue

Vaccines are defined once in a network-wide catalogue; a dose given to a pet is a VACCINATION_RECORD that points at a catalogue entry and at the appointment where it was given. Vets record doses only at appointments assigned to them. A dose with no planned follow-up has no due date and is never overdue (Section 8.3).

6.1 GET /api/v1/vaccines

Returns the vaccine catalogue, alphabetical by vaccineName. There is no per-shelter catalogue — every shelter sees the same list. Not paginated.

Request

Field	Value
Method	GET
Auth required	Yes — Admin, Staff, Veterinarian
Pending allowed	No
Query params	name (optional — case-insensitive contains match on vaccineName)

Example request

```
GET /api/v1/vaccines?name=rab
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Vaccines retrieved successfully",
  "data": [
    {
      "vaccineID": 2,
      "vaccineName": "Rabies",
      "manufacturer": "Example Biologics",
      "vaccineDesc": "Core vaccine. Annual booster."
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted (Adopter, Volunteer)	FORBIDDEN	403

6.2 POST /api/v1/vaccines

Adds a vaccine to the network-wide catalogue.

Request

Field	Value
Method	POST
Auth required	Yes — Veterinarian, Admin
Pending allowed	No
Body fields	vaccineName (string, required, ≤45), manufacturer (string, optional, ≤45), vaccineDesc (string, optional, ≤500)

Optional fields that are blank or whitespace-only are stored as null. Uniqueness is on vaccineName **plus** manufacturer, compared case-insensitively, so the same vaccine from two manufacturers is two entries.

Example request

```
POST /api/v1/vaccines
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "vaccineName": "DHPP",
  "manufacturer": "Example Biologics",
  "vaccineDesc": "Distemper, hepatitis, parvovirus, parainfluenza."
}
```

Response

```
201 Created
{
  "success": true,
  "message": "Vaccine created successfully",
```

```
  "data": {
    "vaccineID": 5,
    "vaccineName": "DHPP",
    "manufacturer": "Example Biologics",
    "vaccineDesc": "Distemper, hepatitis, parvovirus, parainfluenza."
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
vaccineName missing or blank, or any field of the wrong type	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
A vaccine with the same name and manufacturer already exists (case-insensitive)	CONFLICT	409
vaccineName or manufacturer over 45 chars, or vaccineDesc over 500 chars	VALIDATION_ERROR	422

6.3 PUT /api/v1/vaccines/:id

Partially updates a catalogue entry. Same field set and rules as 6.2.

Request

Field	Value
Method	PUT
Auth required	Yes — Veterinarian, Admin
Pending allowed	No
Path params	id (int, required) — vaccineID
Body fields	Any subset of: vaccineName (≤45, cannot be blank), manufacturer (≤45), vaccineDesc (≤500). Sending "" or null for manufacturer or vaccineDesc clears it.

Example request

```
PUT /api/v1/vaccines/5
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "vaccineDesc": null }
```

Response

```
200 OK
{
  "success": true,
  "message": "Vaccine updated successfully",
  "data": {
    "vaccineID": 5,
    "vaccineName": "DHPP",
    "manufacturer": "Example Biologics",
    "vaccineDesc": null
  }
```

```
}
```

> ***Edits are network-wide:*** *doses already given keep pointing at the vaccine's vaccineID, so a rename or correction appears immediately on every pet's passport at every shelter. Because the catalogue is shared, any vet or admin can change an entry that other shelters are already using — correct the entry rather than creating a near-duplicate.*

Error Cases

Scenario	Error Code	HTTP Status
Empty body, id not a positive integer, wrong field type, or vaccineName blank	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Vaccine does not exist	NOT_FOUND	404
The new name and manufacturer match another existing vaccine (case-insensitive)	CONFLICT	409
A field is over its length limit	VALIDATION_ERROR	422

6.4 POST /api/v1/appointments/:id/vaccinations

Records a vaccine dose given at an appointment.

Request

Field	Value
Method	POST
Auth required	Yes — Veterinarian only (assigned vet)
Pending allowed	No
Path params	id (int, required) — appointmentID
Body fields	vaccineID (int, required), administeredDate (date, required, not in the future), dueDate (date, optional or null — must be after administeredDate)

petID, administeredBy (the calling vet), administeredAt (the appointment's shelter) and appointmentID are set server-side and cannot be sent. `dueDate` null or omitted means no further dose is planned, so the dose is never overdue. A dose may be recorded against an appointment in **any** status except Cancelled — including one that is past-dated or already Completed — because vets often log doses after the visit.

Example request

```
POST /api/v1/appointments/120/vaccinations
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "vaccineID": 2,
  "administeredDate": "2026-09-28",
  "dueDate": "2027-09-28"
}
```

Response

```
201 Created
{
  "success": true,
  "message": "Vaccination recorded successfully",
  "data": {
    "recordID": 31,
    "petID": 9,
    "vaccineID": 2,
    "vaccine": { "vaccineName": "Rabies", "manufacturer": "Example
Biologics" },
    "administeredDate": "2026-09-28T00:00:00Z",
    "dueDate": "2027-09-28T00:00:00Z",
    "administeredBy": 7,
    "administeredAt": 3,
    "appointmentID": 120
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
vaccineID or administeredDate missing, or any field malformed	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Caller is Staff or Admin	FORBIDDEN	403
Appointment does not exist or is assigned to another vet; or vaccineID does not exist	NOT_FOUND	404
Appointment is Cancelled	CONFLICT	409
administeredDate is in the future	VALIDATION_ERROR	422
dueDate is not after administeredDate	VALIDATION_ERROR	422

6.5 GET /api/v1/appointments/:id/vaccinations

Lists the doses recorded at one appointment, oldest first.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian (assigned vet), Staff (same shelter), Admin (any)
Pending allowed	No
Path params	id (int, required) — appointmentID

Example request

```
GET /api/v1/appointments/120/vaccinations
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Vaccination records retrieved successfully",
  "data": [
    {
      "recordID": 31,
      "petID": 9,
      "vaccineID": 2,
      "vaccine": { "vaccineName": "Rabies", "manufacturer": "Example
Biologics" },
      "administeredDate": "2026-09-28T00:00:00Z",
      "dueDate": "2027-09-28T00:00:00Z",
      "administeredBy": 7,
      "administeredAt": 3,
      "appointmentID": 120
    }
  ]
}
```

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Staff caller and the appointment is at another shelter; or a role other than Veterinarian, Staff, Admin	FORBIDDEN	403
Appointment does not exist, or (for a vet) is assigned to another vet	NOT_FOUND	404

6.6 GET /api/v1/vets/me/vaccinations/overdue

Returns the overdue-vaccination worklist for the vet's shelter: every active pet whose most recent dose of some vaccine is past its due date. The rules are in Section 8.3.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Path/Body fields	None — shelter resolved from the vet's own record

Example request

```
GET /api/v1/vets/me/vaccinations/overdue
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Overdue vaccinations retrieved successfully",
```

```
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "petPhoto": "...",
      "vaccineID": 2,
      "vaccineName": "Rabies",
      "doseNumber": 2,
      "dueDate": "2026-09-20T00:00:00Z",
      "daysOverdue": 13
    }
  ]
}
```

Sorted most overdue first. `doseNumber` is the next dose due — the number of doses already given for that pet and vaccine, plus one. A vet with no shelter assigned receives an empty array. Not paginated.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

6.7 GET /api/v1/vets/me/stats

Returns the vet's treatment statistics.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Path/Body fields	None

Example request

```
GET /api/v1/vets/me/stats
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Stats retrieved successfully",
  "data": { "petsTreated": 14 }
}
```

`petsTreated` is the number of **distinct pets** across the vet's past, non-cancelled appointments. A pet seen at five appointments counts once. "Past" is judged by appointmentDate, consistent with the displayed status in Section 8.1.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

7. Pets, Health Passport & Health Records

Vets read the pets at their own shelter, and read and write the clinical history that follows each pet. The health passport is **universal**: it is keyed by pet, not by shelter, so it travels with the pet when it is transferred between shelters (7.3).

7.1 GET /api/v1/vets/me/pets

Returns the pets at the vet's shelter, regardless of adoption status, optionally filtered. Mirrors Staff GET /staff/me/pets (11-Staff API Reference, 1.1) with one extra filter.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Query params	adoptionStatus (optional — incoming / available / adopted / fostered / transferred / deceased), species (repeatable speciesID), breed (repeatable breedName), size (repeatable — Small / Medium / Large), minAge / maxAge (int, months, inclusive), petName (case-insensitive contains match), sort (optional — "newest" orders by intakeDate descending; default is petID descending), page (int, optional, default 1), limit (int, optional, default 20)

Example request

```
GET /api/v1/vets/me/pets?
adoptionStatus=available&size=Medium&petName=bud&sort=newest
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Pets retrieved successfully",
  "data": [
    {
      "petID": 9,
      "petName": "Buddy",
      "petAge": "2 yr",
      "petSex": "M",
      "petPhoto": "...",
      "breed": { "...": "..." },
      "adoptionStatus": "available"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

A vet with no shelter assigned receives an empty list rather than an error — unlike Staff, who receive 409 (Appendix A).

Error Cases

Scenario	Error Code	HTTP Status
Invalid adoptionStatus, size, age bound, sort, or pagination value	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

7.2 GET /api/v1/vets/me/pets/:id

Returns full detail for one pet at the vet's shelter, in the Staff pet-detail shape (11-Staff API Reference, 1.2): microchipID, intakeDate, intakeType and the other staff-only fields on top of the public pet profile.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Path params	id (int, required) — petID

Example request

```
GET /api/v1/vets/me/pets/9
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Pet retrieved successfully",
  "data": {
    "petID": 9,
    "petCode": "PE000009",
    "petName": "Buddy",
    "petDOB": "2024-02-14",
    "petAge": "2 yr",
    "petSize": "Medium",
    "petBGroup": "N/A",
    "microchipID": "982000123456789",
    "intakeDate": "2026-05-02",
    "intakeType": "stray",
    "featuredFlag": false,
    "...": "...same fields as public GET /pets/:id"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer	BAD_REQUEST	400

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Pet does not exist, or is at another shelter (a different shelter's pet is answered as not found)	NOT_FOUND	404

7.3 GET /api/v1/vets/me/pets/:id/health-passport

Returns a pet's universal health passport: its detail, every health record, every vaccine dose, and its full transfer history.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian only
Pending allowed	No
Path params	id (int, required) — petID

Example request

```
GET /api/v1/vets/me/pets/9/health-passport
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Health passport retrieved successfully",
  "data": {
    "pet": { "...": "same shape as GET /vets/me/pets/:id (7.2)" },
    "aiSummaryAvailable": true,
    "healthRecords": [
      {
        "recordID": 18,
        "appointmentID": 120,
        "appointmentCode": "APT-00120",
        "createdAt": "2026-09-28T11:05:00Z",
        "recordDesc": "Healthy weight, no concerns. Rabies booster given.",
        "vetName": "Dr. Jay Asarathi",
        "shelterName": "Athens Shelter"
      },
      {
        "recordID": 12,
        "appointmentID": null,
        "appointmentCode": null,
        "createdAt": "2026-06-10T10:00:00Z",
        "recordDesc": "Dental check, mild tartar.",
        "vetName": null,
        "shelterName": "Decatur Shelter"
      }
    ],
```

```
      "vaccinations": [
        {
          "recordID": 31,
          "vaccineName": "Rabies",
          "administeredDate": "2026-09-28T00:00:00Z",
          "dueDate": "2027-09-28T00:00:00Z",
          "status": "Up to Date"
        },
        {
          "recordID": 27,
          "vaccineName": "DHPP",
          "administeredDate": "2025-10-20T00:00:00Z",
          "dueDate": "2026-10-20T00:00:00Z",
          "status": "Due Soon"
        },
        {
          "recordID": 22,
          "vaccineName": "Bordetella",
          "administeredDate": "2025-09-20T00:00:00Z",
          "dueDate": "2026-09-20T00:00:00Z",
          "status": "Overdue"
        },
        {
          "recordID": 15,
          "vaccineName": "Distemper",
          "administeredDate": "2025-05-10T00:00:00Z",
          "dueDate": null,
          "status": "No Further Dose"
        }
      ],
      "transferHistory": [
        {
          "recordID": 2,
          "transferDate": "2026-08-15T09:00:00Z",
          "transferReason": "Capacity",
          "transferStatus": "Completed",
          "fromShelterName": "Decatur Shelter",
          "toShelterName": "Athens Shelter",
          "fromStaffName": "Casey Example",
          "toStaffName": "Alex Placeholder"
        }
      ]
    }
  }
```


> ***Universal by design:*** *health records, vaccine doses and transfers are read by petID only — never filtered by the shelter the vet works at. A vet at a pet's current shelter therefore sees history created at every shelter the pet has previously been at (the Decatur Shelter note above). This is the point of the passport. The pet itself must still be at the vet's shelter now; a vet whose shelter the pet has left no longer has access.*

Each healthRecords item carries appointmentID and appointmentCode when the note was written at an appointment (including completion notes, 5.3), and null for both on standalone notes (7.4). vetName is null when the author's account has since been deleted — clients show "Former vet" (Section 9). The dose status values are defined in Section 8.2. `aiSummaryAvailable` is true when the AI provider is configured on the server (worked out from configuration alone, with no AI call); the client uses it to enable the "Summarize with AI" button, which stays disabled when it's false.

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Pet does not exist, or is not at the vet's shelter	NOT_FOUND	404

7.4 POST /api/v1/pets/:id/health-records

Adds a standalone health note to a pet — a note not tied to any appointment.

Request

Field	Value
Method	POST
Auth required	Yes — Veterinarian only
Pending allowed	No
Path params	id (int, required) — petID
Body fields	recordDesc (string, required, ≤500 chars)

vetID is set to the calling vet. The pet must be at the vet's shelter. Unlike the read routes in 7.2 and 7.3, a pet at another shelter returns 403 here, not 404.

Example request

```
POST /api/v1/pets/9/health-records
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "recordDesc": "Limping on left hind leg after play. Rest for 5 days." }
```

Response

```
201 Created
{
  "success": true,
  "message": "Health record created successfully",
  "data": {
    "recordID": 19,
    "petID": 9,
    "appointmentID": null,
    "appointmentCode": null,
    "recordDesc": "Limping on left hind leg after play. Rest for 5 days.",
    "createdAt": "2026-10-03T09:15:00Z",
    "vetName": "Dr. Jay Asarathi",
    "shelterName": "Athens Shelter"
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
recordDesc missing, blank or whitespace-only, over 500 chars, or id not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Pet is at another shelter	FORBIDDEN	403
Pet does not exist	NOT_FOUND	404

7.5 PUT /api/v1/health-records/:id

Edits the text of a health record. Only the vet who wrote it may edit it.

Request

Field	Value
Method	PUT
Auth required	Yes — Veterinarian only (author)
Pending allowed	No
Path params	id (int, required) — recordID
Body fields	recordDesc (string, required, ≤500 chars) — the only editable field

This applies to appointment-completion notes (5.3) as well as standalone notes. A record with no vet (its author's account was deleted) cannot be edited by anyone, because no caller can be its author.

Example request

```
PUT /api/v1/health-records/19
Authorization: Bearer <accessToken>
Content-Type: application/json

{ "recordDesc": "Limping on left hind leg after play. Rest for 7 days." }
```

Response

```
200 OK
{
  "success": true,
  "message": "Health record updated successfully",
  "data": { "...": "same shape as the POST /pets/:id/health-records response
(7.4)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
recordDesc missing, blank, or over 500 chars; any other field sent; or id not a positive integer	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
The record was written by a different vet, or has no vet	FORBIDDEN	403

Scenario	Error Code	HTTP Status
Health record does not exist	NOT_FOUND	404

7.6 GET /api/v1/pets/:id/health-passport/summary

Returns an AI summary of the pet's health passport plus "Points to review", each citing the health record, vaccination dose or transfer it came from. Overdue-vaccine points always come from the vaccination rules, never the AI. The summary is cached per pet until a record, dose or transfer changes, so reopening it (or another user opening it) doesn't call the AI again. Adding a health record changes the passport, so the next request regenerates the summary.

Request

Field	Value
Method	GET
Auth required	Yes — Veterinarian (also open to Staff and Admin; see 11-Staff API Reference)
Pending allowed	No
Path params	id (int, required) — petID, a positive integer

A vet can only summarise pets at their own shelter. A pet at another shelter, or any pet when the vet has no shelter, returns 404 (not 403), so another shelter's pets aren't revealed — the same masking as 7.3.

Example request

```
GET /api/v1/pets/9/health-passport/summary
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Health passport summary retrieved successfully",
  "data": {
    "aiAvailable": true,
    "summary": "<2–4 sentence summary>",
    "pointsToReview": [
      { "text": "Rabies (Dose 2) is 12 days overdue.", "source": { "type":
"vaccination", "recordID": 41 }, "origin": "rules" },
      { "text": "<a follow-up the notes mention>", "source": { "type":
"healthRecord", "recordID": 87 }, "origin": "ai" }
    ],
    "generatedAt": "2026-10-12T14:03:00.000Z"
  }
}
```

`source.type` is "healthRecord", "vaccination" or "transfer"; `origin` is "rules" or "ai". Rule points (overdue vaccines) come first. A pet with nothing on file gets the fixed summary "No health records, vaccinations or transfers are on file for this pet yet." with no AI call and `generatedAt` null. If the AI is not configured, the response is still 200 with message "AI is not configured" and data `{ "aiAvailable": false, "summary": null, "pointsToReview": [], "generatedAt": null }`.

Error Cases

Scenario	Error Code	HTTP Status
id is not a positive integer — "id must be a positive integer"	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Pet does not exist ("No pet exists with ID <id>"), or is not at the vet's shelter	NOT_FOUND	404
The AI provider failed or returned nothing usable — "The AI summary is unavailable right now — try again shortly". Retryable; nothing is cached	SERVICE_UNAVAILABLE	503

For the full response shape, the summary rules and caching, see 15-AI Features API Reference.

8. Status & Derived Values

Several values the API returns are computed rather than stored. This section defines them so clients do not re-derive them.

8.1 Appointment status: displayed vs stored

An appointment has a stored `appointmentStatus` (Scheduled, Completed or Cancelled). The `status` field returned in lists and details is the **displayed** status, derived from the stored value and the appointment date. The vet detail endpoint (5.2) returns both.

Stored appointmentStatus	appointmentDate	Displayed status
Scheduled	In the future	Scheduled
Scheduled	In the past	Completed
Completed	Any	Completed
Cancelled	Any	Cancelled

- **Display is a convenience, not a clinical fact.** A past-dated Scheduled appointment displays as Completed even though no vet has recorded anything. The vet completes it explicitly with PATCH /appointments/:id/status (5.3), and that endpoint checks the **stored** value, so a past-dated Scheduled appointment can still be completed.
- **Queue filters follow the displayed status.** upcoming=true returns Scheduled appointments in the future; upcoming=false returns past-dated or Cancelled ones (5.1).
- **Edit and cancel follow the stored status and date.** Only a stored Scheduled appointment whose date has not passed can be edited (5.4) or cancelled (5.5).
- **Double-booking guard.** A unique constraint on (petID, vetID, appointmentDate), applied to Scheduled appointments only, stops the same pet being booked twice with the same vet at the same time. Cancelled and Completed appointments do not hold the slot.

8.2 Vaccination status

Each dose on the health passport (7.3) carries a `status` computed from its `dueDate` at read time:

Status	Condition
Overdue	dueDate is in the past
Due Soon	dueDate is today or within the next 30 days
Up to Date	dueDate is more than 30 days away

Status	Condition
No Further Dose	dueDate is null — no follow-up dose is planned

This status is computed per dose row, so an older dose that has since been superseded by a newer one can still read Overdue on the passport. The overdue worklist (8.3) is the authoritative answer to "what is actually overdue".

8.3 Overdue logic

GET /vets/me/vaccinations/overdue (6.6) applies the following rules:

- **Scope:** active pets at the vet's shelter only — pets whose adoptionStatus is adopted or deceased are excluded.
- **Latest dose only:** for each pet and vaccine pair, only the most recent dose counts. If a newer dose has been recorded, the older dose's passed due date is irrelevant.
- **Overdue means:** that latest dose has a dueDate in the past. A null dueDate is never overdue.
- **doseNumber** is the number of doses given to that pet for that vaccine, plus one — the dose that is now due.
- **daysOverdue** is the whole number of days between the due date and today.
- **Ordering:** most overdue first.

9. Data Retention on Account Deletion

A vet can leave in two ways (4.7). Deactivating keeps everything; deleting removes the vet's identity but deliberately preserves their clinical history, because pets' medical records outlive the people who wrote them.

Data	On deactivate	On delete
Veterinarian row	Kept — accountStatus set to Deactivated	Removed
Users row (login identity)	Kept	Removed
Refresh token	Cleared (forces logout)	Gone with the Users row
Government ID row and its Storage file	Kept	Both removed
Appointment.vetID	Unchanged	Set to NULL
HealthRecord.vetID	Unchanged	Set to NULL
VaccinationRecord.administeredBy	Unchanged	Set to NULL

The three history columns are foreign keys declared `ON DELETE SET NULL`, so the database itself preserves the appointments, notes and doses when the vet row disappears; nothing depends on application code remembering to do so. Consequences clients must handle:

- **"Former vet".** Wherever a vet is shown by name — vetName in appointment details, health records, the passport — the value is null for a deleted vet. Clients render "Former vet".
- **Notes become permanent.** A health record whose vet is null can be edited by nobody (7.5), since no caller can be its author.
- **Nothing future is orphaned.** Both modes are refused while the vet has any future Scheduled appointment (4.7), so deletion never leaves a booked patient without a vet. Only past appointments end up with a null vetID.

10. Testing Coverage

The vet portal is covered by two layers: fast unit tests with Prisma fully mocked, and one integration journey file that exercises the real HTTP stack against the test database.

10.1 Unit tests

Written with Jest and Supertest. Prisma is fully mocked, so these tests verify routing, role and Pending-state gating, validation and error mapping, and the shape of each response without touching a database.

Suite	Covers
unit/vet/vets.selfService	Section 4: profile read/update, E.164 phone handling, rejected fields, government ID submit/resubmit and masking, onboarding step and completion rules, deactivate and delete including the upcoming-appointment block
unit/vet/appointments	Section 5: own-queue scoping and filters, detail with both status fields, completion rules and notes, restricted edit, 404 masking of other vets' appointments, Staff/Admin 403s
unit/vet/vaccinations	Section 6: catalogue list/create/edit and duplicate rules, recording doses with server-set fields and date validation, per-appointment listing and access, overdue worklist, stats
unit/vet/petsHealthRecords	Section 7: shelter-scoped pet list and detail, universal health passport, standalone health records, author-only editing
unit/vet/overview	Vet overview figures, including petsTreated (6.7)
unit/auth/login	Pending vet can log in and receives accountStatus, onboardingComplete and onboardingStep; Pending Volunteer and Admin accounts are still refused with 401
unit/staff/vets.management	Manager approval of vets (2.3): the 409 when onboarding is incomplete or the government ID is not Verified, decline at any stage, deactivation
unit/staff/healthPassportSummary	Section 7 (7.6): access (Staff at another shelter → 403; Veterinarian at another shelter or with no shelter → 404; unknown pet → 404; bad id → 400), AI not configured → aiAvailable false with no AI call, empty passport → fixed summary, cache (cold → one AI call and cached; same input → no call; a new record changes the hash → regenerated; current overdue points added even from cache; none for an adopted pet), AI error or empty AI summary → 503 with nothing cached, the AI input carries no people's names or due dates, reply validation drops uncited, unknown and repeated sources

10.2 Integration journeys

All three journeys live in a single file, `integration/vet/sprint6Journeys.test.js`. They send real HTTP requests to the app running against the test database. Test data is created through API calls (not inserted directly), and everything is removed through Prisma in `afterAll`. The file must be run with `--runInBand` so the journeys execute serially, since they share one database and depend on state built up step by step.

```
npx jest integration/vet/sprint6Journeys.test.js --runInBand
```

Journey 1 — Vet onboarding and approval

1. Register as a vet at a shelter that has a manager.
2. Log in while Pending; the response carries accountStatus Pending, onboardingComplete false and the current onboardingStep.
3. Confirm a non-allowPending route (for example GET /vets/me/appointments) returns 401.
4. Work through the wizard: PUT /vets/me with personal and address data, PATCH onboarding-step for steps 2 and 3, POST the government ID, PATCH onboarding-step for step 4, then PATCH onboarding-complete.
5. As the shelter Manager, attempt approval before the ID is Verified and receive 409 naming the missing item.
6. The Manager verifies the government ID, then approves the vet.
7. The vet logs in again as Active and can now reach the full portal.

Journey 2 — Appointment, vaccination and completion

1. Staff create an appointment for a pet with the vet (POST /appointments).

2. The vet sees it in GET /vets/me/appointments.
3. The vet records a vaccination against it (POST /appointments/:id/vaccinations).
4. The vet completes it with notes (PATCH /appointments/:id/status). Completion succeeds only once appointmentDate has been reached (5.3).
5. The detail response shows status Completed, the dose under vaccinesAdministered, and the notes as a health record.

Journey 3 — Universal passport across a transfer

1. A vet at shelter A records a dose for a pet at shelter A.
2. The pet is transferred from shelter A to shelter B and the transfer is approved (11-Staff API Reference, Section 5).
3. A vet at shelter B reads the pet's health passport and sees the dose that was recorded at shelter A, alongside the transfer in transferHistory.
4. The vet at shelter A now receives 404 for the same pet, since it is no longer at their shelter.

Appendix A — Relationship to the Staff Reference (11)

The vet onboarding and close-account flows deliberately mirror the Staff Portal flows in 11-Staff API Reference, Section 12: the same wizard shape and step arithmetic, the same government ID contract, and the same deactivate/delete modes. Where this document says an endpoint behaves "like Staff", the Staff reference is authoritative for anything not restated here. The differences are:

Topic	Staff (11)	Veterinarian (12)
Route prefix	/staff/me/…	/vets/me/…
Wizard steps	2 Personal, 3 Address, 4 Identity, 5 Review	Identical
Pending access	Pending staff can call 12.1, 12.2, 12.4–12.7 and log out; everything else is 401	Same mechanism (allowPending, 401 not 403); the allowed set also excludes DELETE /vets/me (2.2)
Who approves	A shelter's Manager approves staff; a Pending Manager is approved by an Admin	Always the shelter's Manager (11.5); registration is refused if the shelter has no manager (2.1)
Approval gate	Onboarding complete and ID Verified (11.3)	Same gate applied to vets (2.3)
Editable via PUT	avatarSeed, staffName, staffPhone, staffDOB, staffSex; shelterID, staffDesignation, accountStatus rejected	avatarSeed, vetName, vetPhone, vetDOB, vetSex, address fields; shelterID, accountStatus rejected
Government ID	*****6789 mask, resubmit when Rejected	Identical contract; userType "Veterinarian"
What blocks closing the account	A Pending application still assigned to them	Any future Scheduled appointment
Side effects of closing	Clears managerStaffID on shelters they manage	History columns set to NULL on delete (Section 9)
Pet at another shelter	403 FORBIDDEN on detail and health passport	404 NOT_FOUND on the vet read routes (7.2, 7.3, 7.6); 403 only on POST /pets/:id/health-records
No shelter assigned	409 CONFLICT on pet list	Empty list (7.1, 6.6)
Appointments	Shelter-wide queue; create, edit, cancel	Own queue only; complete, and edit date and reason; no create or cancel (5.5)
Other vet's appointment	Not applicable	404, never 403 (2.5)

> ***Note for 11-Staff API Reference:*** *the Manager-side vet endpoint (11.5, PATCH /staff/me/vets/:id/status) does not currently list the 409 for approving a vet whose onboarding is incomplete or whose government ID is not Verified. That rule is specified here (2.3) and the Staff reference should be updated to match.*